Facultad de ingeniería

Docente

Leidy Mariani Dorado Flórez

Materia

Sistemas Distribuidos

Trabajo

Práctica 4: Gestión de imágenes y contenedores

Estudiantes

Jhonn Alexander Pantoja Jimenez

Popayán, Cauca 05 de marzo de 2026

PARTE 1 - Exploración de imágenes..... 3
Actividad 1: Ver imágenes locales..... 3
Actividad 2: Descargar nuevas imágenes.....4
PARTE 2 - Gestión de Contenedores..... 5
Actividad 3: Crear un contenedor..... 5
Actividad 4: Administrar el contenedor..... 6
PARTE 3: Análisis conceptual..... 7
PARTE 4: Reto en equipo..... 7
PARTE 5: Análisis Crítico..... 8
CONCLUSIÓN..... 9

PARTE 1 - Exploración de imágenes

Actividad 1: Ver imágenes locales

Usar Docker images

```
PROBLEMAS    SALIDA    TERMINAL    DEVDB    PUERTOS    CONSOLA DE DEPURACIÓN

PS G:\Distribuidos\apiclase> docker images

IMAGE                  ID            DISK USAGE   CONTENT SIZE   EXTRA
apiclase:latest        8d2fe39704a8       237MB         62.9MB
perfil:latest          4904bcb764c4       237MB         62.9MB
serviciologin:latest   c25afc2633d0       237MB         62.9MB
PS G:\Distribuidos\apiclase>
```

- **Image ID:**
  Es el identificador único de una imagen
- **TAG:**
  Es el nombre, etiqueta o versión de la imagen
- **SIZE**
  Corresponde al tamaño de la imagen
- **¿Alguna se está ejecutando?**
  No, se puede apreciar que todas están inactivas.

Actividad 2: Descargar nuevas imágenes

- Se ejecutó docker pull postgres

```
PS G:\Distribuidos\apiclase> docker pull postgres
Using default tag: latest
latest: Pulling from library/postgres
3facd37768c9: Downloading [=========================>                        ]  3.146MB/5.957MB
90f0f713dfa1: Downloading [==================================================>]  52.46kB/52.46kB
56a239f8dc77: Pull complete
be4ad6e904ed: Pull complete
e7dee01ee448: Pull complete
967e70a4ad93: Pull complete
3859b312d253: Pull complete
70f3de31936a: Pull complete
358db71ef364: Pull complete
1b030cb6bea0: Pull complete
0480b5ae6db9: Pull complete
4e2113a73362: Pull complete
e232269e9149: Pull complete
eea32e6f70e8: Pull complete
Digest: sha256:69e8582b781cb44fa4557b98ed586fe68361e320d9b12f9707494335634f4f3d
Status: Downloaded newer image for postgres:latest
docker.io/library/postgres:latest
PS G:\Distribuidos\apiclase>
```

- Luego Docker images:

```
IMAGE                 ID            DISK USAGE   CONTENT SIZE   EXTRA
apiclase:latest       8d2fe39704a8       237MB         62.9MB
perfil:latest         4904bcb764c4       237MB         62.9MB
postgres:latest       69e8582b781c       649MB          168MB
serviciologin:latest  c25afc2633d0       237MB         62.9MB
PS G:\Distribuidos\apiclase>
```

- ¿Qué cambió?
  Se ha creado una nueva imagen llamada postgres:latest

- ¿El servicio ya está funcionando?
  No, solo se ha descargado la imagen, para que funcione el servicio se debe crear y correr un contenedor creado a partir de esa imagen.

PARTE 2 - Gestión de Contenedores

Actividad 3: Crear un contenedor

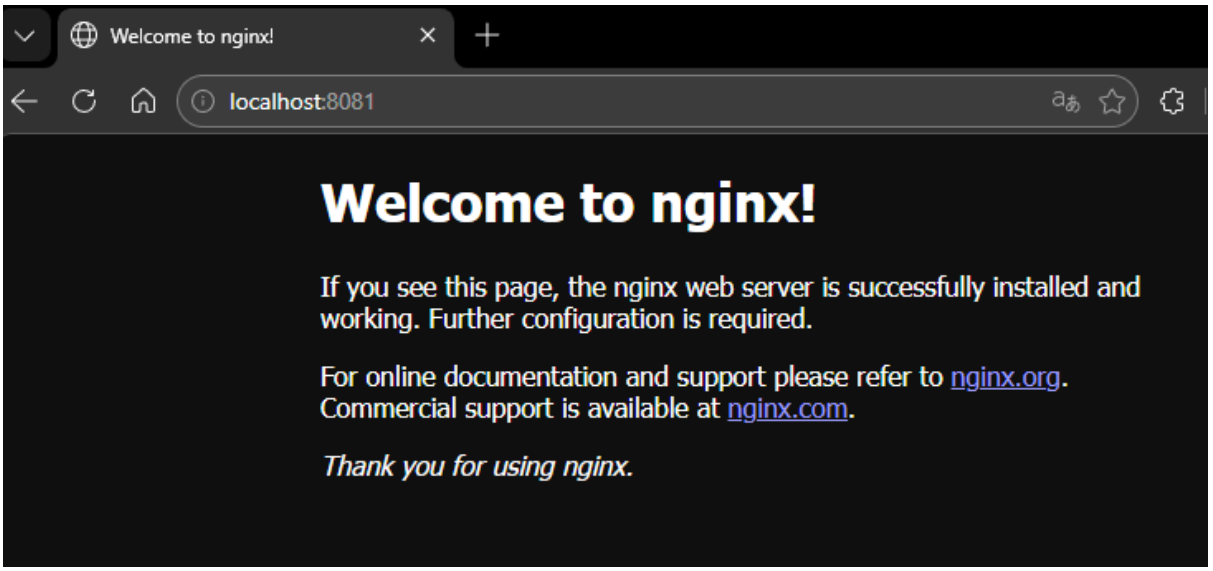
- Contenedor: *docker run -d --name servidor-web -p 8081:80 nginx*

```
PS G:\Distribuidos\apiclase> docker run -d --name servidor-web -p 8081:80 nginx
3ea8cb28d9baf348f1974f2b4f62d1b5ae3e77500fa401457b22244bbc1a01cc
PS G:\Distribuidos\apiclase>
```

- Se verifica con: *docker ps*

```
PS G:\Distribuidos\apiclase> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                      NAMES
3ea8cb28d9ba   nginx     "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   servidor-web
PS G:\Distribuidos\apiclase>
```

- En el navegador vamos a: *http://localhost:8081*



Actividad 4: Administrar el contenedor

- Detener el contenedor:

```
PS G:\Distribuidos\apiclase> docker stop servidor-web
servidor-web
PS G:\Distribuidos\apiclase>
```

- Ver todos los contenedores:

```
PS G:\Distribuidos\apiclase> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS                      PORTS     NAMES
3ea8cb28d9ba   nginx     "/docker-entrypoint.…"   4 minutes ago   Exited (0) 44 seconds ago             servidor-web
PS G:\Distribuidos\apiclase>
```

- Reiniciar contenedor:

```
PS G:\Distribuidos\apiclase> docker start servidor-web
servidor-web
PS G:\Distribuidos\apiclase>
```

- Eliminar contenedor:

```
PS G:\Distribuidos\apiclase> docker rm -f  servidor-web
servidor-web
PS G:\Distribuidos\apiclase>
```

PARTE 3: Análisis conceptual

- Completar tabla:

Concepto	Imagen	Contenedor
¿Es ejecutable?	No, es una plantilla.	Si se ejecuta a partir de una imagen
¿Ocupa recursos en ejecución?	Solo espacio en disco	Si ocupa los recursos necesarios para que todo funcione correctamente donde se corra.
¿Puede modificarse?	No, tienen versionamientos	Si, de manera temporal durante su ejecución
¿Puede eliminarse?	Si, cuando no está siendo usada por contenedor	Si.

PARTE 4: Reto en equipo

1. Se ejecutó el contendor de una nueva imagen: httpd

```
PS G:\Distribuidos\apiclase> docker run -it --rm -d -p 8080:80 --name conthttpd httpd:latest
d4168c9ab07831cea2fc28801944338d83f12638a14770bdefc159449f47cd2b
```

2. Lo vemos activo:

```
PS G:\Distribuidos\apiclase> docker ps
CONTAINER ID   IMAGE          COMMAND              CREATED        STATUS         PORTS                                       NAMES
d4168c9ab078   httpd:latest   "httpd-foreground"   3 minutes ago  Up 3 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp     conthttpd
PS G:\Distribuidos\apiclase>
```

- ¿Qué tipo de servicio es?
  Es un servicio web
- ¿En qué tipo de sistema distribuido podría usarse?
  En microservicios, aplicaciones en la nube o plataformas web
- ¿Qué pasaría si ese servicio falla?
  Los usuarios no podrían acceder al sitio web o aplicación

PARTE 5: Análisis Crítico

- ¿Cual es la diferencia real entre imagen y contenedor?

Una imagen es el conjunto de archivos y configuraciones necesarios para ejecutar una aplicación mientras que un contenedor es la ejecución de esa imagen, es decir, cuando la aplicación realmente está funcionando en el sistema.

- ¿Por qué docker facilita la escalabilidad?

Docker permite crear varios contenedores de una misma aplicación de manera rápida lo que ayuda a que un sistema pueda manejar más usuarios o solicitudes cuando la demanda aumenta.

- ¿Cómo se relaciona esto con los microservicios?

En los microservicios las aplicaciones se dividen en varios servicios comunicados entre sí, pero que funcionan de manera independiente y docker lo que permite es ejecutar cada uno de los servicios en su propio contenedor de igual manera funcionan independientes.

- ¿Qué ventajas ofrece frente a instalar software directamente?

Docker permite que las aplicaciones se ejecuten con todo lo que necesitan sin afectar al resto del sistema. Además, facilita migrar aplicaciones entre diferentes computadores o servidores sin tener que reinstalar todo nuevamente. En sistemas distribuidos también permite crear varios contenedores a partir de una misma imagen, lo que ayuda a mantener el servicio disponible si alguno falla.

CONCLUSIÓN

Hemos podido comprender el funcionamiento básico de Docker entendiendo muy bien la diferencia que hay entre las imágenes y los contenedores. Hemos aprendido la importancia que tiene el uso de la terminal creando y administrando tanto imágenes como contenedores por medio de comandos, además, vemos como los contenedores permiten correr diferentes servicios al mismo tiempo en una sola máquina de manera aislada cada una usando su propio puerto lo que lo hace muy útil para el desarrollo de servicios en sistemas distribuidos, como también descargar imagenes que ya están preestablecidas.